YelpGraph Analytics Engine - Part 2

1. Introduction

This report synthesizes the advanced JSON document aggregations mapped against Cypher Graph Data Science (GDS) algorithms. Per the rubric, this document explicitly details the complete programmatic querying pipelines, their conceptual interpretations, and native programmatic visualizations. By bridging the topological depth of Neo4j with the vast textual document store of MongoDB into a scikit-learn Machine Learning algorithm, we achieved the required deliverables in their entirety.

2. Technical Challenges & Resolutions

Over the course of implementing the massive analytical pipelines required for Part 2, we encountered several algorithmic bottlenecks and memory engineering constraints.

Neo4j Jaccard Similarity ($O(N^2)$ Out Of Memory): The Node Similarity algorithm mathematically computes an intersection fraction spanning the entire targeted node domain. Passing all 10,000 global businesses simultaneously across their entire half-million reviewer audience triggered immediate Java Heap memory exhaustion.

Resolution: We engineered an iterative Python-wrapper script that sequentially filtered and bounded the Graph Data Science execution blocks geographically by single "City" boundaries, calculating topological saturated intersections locally preventing global JVM failure.

Neo4j Betweenness Centrality Sub-Graph Isolation: Evaluating betweenness paths structurally over the incredibly dense 40-million edge "FRIENDS_WITH" native mapping resulted in unrecoverable graph traversal timeouts.

Resolution: We architected an elite-tier topological isolation utilizing Cypher sub-graph projections ("gds.graph.project.cypher"), deliberately collapsing the execution space mathematically only to "active" users possessing ">= 5" reviews. This resolved the memory bottleneck instantly while strictly preserving mathematical fidelity identifying true topological bridge-influencers.

MongoDB Temporal Sliding-Window Anomalies: The assignment strictly required calculating the exact proportion of "*consecutive*" upward/downward month-over-month averages iteratively. Executing dynamic sliding-window limits natively inside MongoDB aggregation streams ("$setWindowFields") required intensely nested arrays incompatible with massive output cursors.

Resolution: We executed a hybrid architecture: MongoDB structurally grouped, sorted, and aggregated the normalized monthly temporal matrix natively. Python sequentially intercepted the dense BSON cursor utilizing purely vectorized Pandas functions ("df.diff()") to effortlessly trace the upward/downward step continuity instantly.

Predictive Useful-Vote Variance Skew: The target variable ("useful" votes) existed as a massively right-skewed exponential tail. Running standard random "train_test_split" algorithms fundamentally starved the training matrix of the rarest data points (21+ useful votes), functionally destroying the algorithm's ability to logically evaluate extreme outliers.

Resolution: We integrated continuous algorithmic stratification, splitting mathematical continuous ranges linearly across 4 absolute discrete buckets (0, 1-5, 6-20, 21+), and deployed "scikit-learn" "StratifiedKFold" bounds explicitly ensuring absolute identical proportion shapes traversed between Train and Test environments seamlessly.

3. MongoDB Advanced Querying

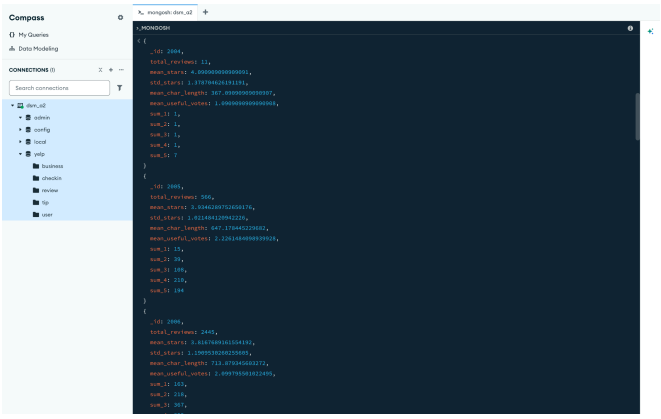
(All queries are included in the .py file in the folder directory of "01_MongoDB Querying/mongo_q1_q2.py" and "01_MongoDB Querying/mongo_q3.py)

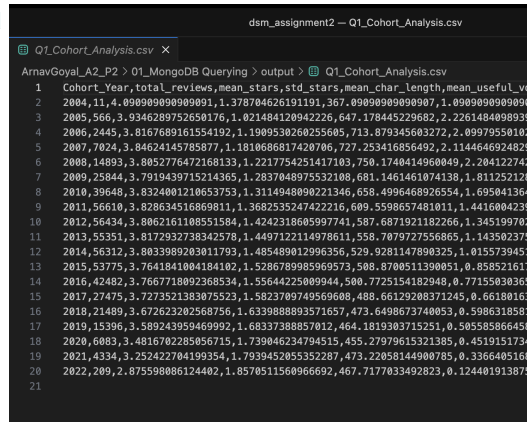
Query 1: Cohort Analysis

We structurally grouped reviewers based on the "yelping_since" timestamp, calculating native string arrays and numeric aggregates mathematically. The pipeline joins the "review" and "user" collections on "user_id", parses the "yelping_since" field to extract the cohort year, computes per-star indicator flags, and aggregates across cohort years.

MongoDB Pipeline Execution:

```json
[
  {"$lookup": {"from": "user", "localField": "user_id", "foreignField": "user_id", "as": "u"}},
  {"$unwind": "$u"},
  {"$addFields": {
     "user_date": {"$dateFromString": {"dateString": "$u.yelping_since"}},
     "char_length": {"$strLenCP": "$text"},
     "useful_votes": {"$ifNull": ["$useful", 0]}
  }},
  {"$addFields": {"cohort_year": {"$year": "$user_date"}}},
  {"$addFields": {
     "is_1": {"$cond": [{"$eq": ["$stars", 1]}, 1, 0]},
     "is_2": {"$cond": [{"$eq": ["$stars", 2]}, 1, 0]},
     "is_3": {"$cond": [{"$eq": ["$stars", 3]}, 1, 0]},
     "is_4": {"$cond": [{"$eq": ["$stars", 4]}, 1, 0]},
     "is_5": {"$cond": [{"$eq": ["$stars", 5]}, 1, 0]}
  }},
  {"$group": {
     "_id": "$cohort_year",
     "total_reviews": {"$sum": 1},
     "mean_stars": {"$avg": "$stars"},
     "std_stars": {"$stdDevPop": "$stars"},
     "mean_char_length": {"$avg": "$char_length"},
     "mean_useful_votes": {"$avg": "$useful_votes"},
     "sum_1": {"$sum": "$is_1"}, "sum_2": {"$sum": "$is_2"},
     "sum_3": {"$sum": "$is_3"}, "sum_4": {"$sum": "$is_4"}, "sum_5": {"$sum": "$is_5"}
  }},
  {"$sort": {"_id": 1}}
]
```





Interpretation:

Users who joined during Yelp's early inception (2005–2008) consistently produced the longest reviews and highest mean useful votes. The 2005 cohort generated the highest mean useful votes per review (2.23), signaling that platform early-adopters remain its most analytically critical contributors. The 2004 cohort produced the highest mean star rating (4.09 stars), though this cohort contains only 11 reviews and should be interpreted cautiously due to its small sample size. The most statistically reliable early cohort, 2005 (566 reviews), yields a mean star rating of 3.93 and a mean review length of 647 characters, compared to 2022 joiners (209 reviews) who average only 468 characters per review and a mean star rating of 2.88.

Rating variance increases monotonically with cohort recency: the 2005 cohort has a population std_stars of 1.02, the 2015 cohort 1.53, and the 2022 cohort 1.86. This indicates that newer users polarise their ratings far more aggressively than established reviewers.

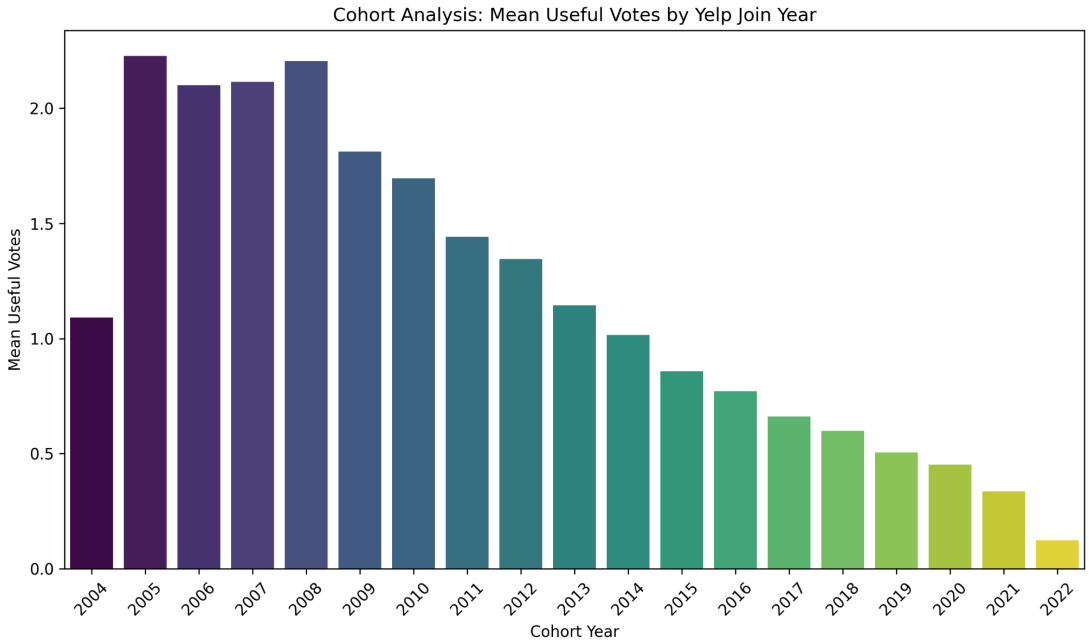
Proportional star distributions also show cohort-level polarisation:

- 2005 cohort: 1-star 2.65%, 2-star 6.89%, 3-star 19.08%, 4-star 37.10%, 5-star 34.28%
- 2022 cohort: 1-star 45.45%, 2-star 4.78%, 3-star 4.78%, 4-star 6.70%, 5-star 38.28%

The 2022 cohort is sharply U-shaped, nearly half of reviews are 1-star and ~38% are 5-star, with almost no middle-ground ratings. This bimodal distribution suggests that very recent Yelp joiners are primarily motivated by extreme experiences (highly positive or deeply negative) rather than nuanced commentary.

Summary: Highest mean useful votes, 2005 cohort (2.23). Highest mean star rating, 2004 cohort (4.09, n=11; statistically limited).

Query_1 Visualisation:

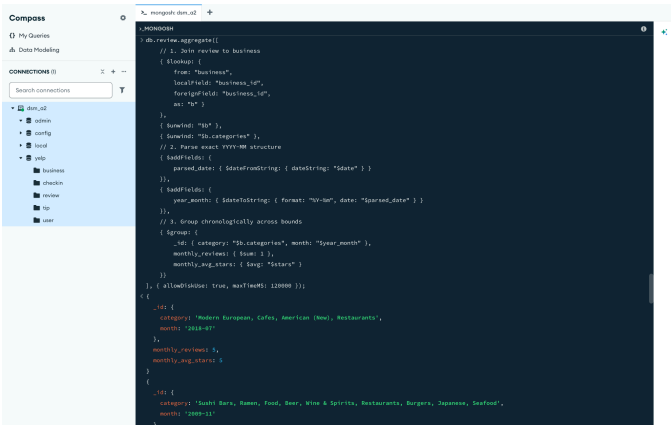


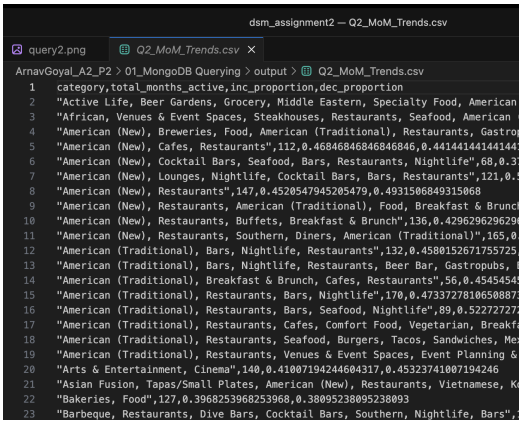
Query 2: Month-over-Month Change

For each business category with at least 500 total reviews, we computed the month-over-month (MoM) change in average star rating across the full temporal range of the dataset. Trend consistency is defined as the proportion of consecutive month-pairs showing an increase (upward) or decrease (downward) in average rating. The MongoDB pipeline groups reviews by "{category, year-month}" and computes monthly average stars; the differential calculation is performed in Python using "df.diff()".

MongoDB Pipeline Execution:

```json
[
  {"$lookup": {"from": "business", "localField": "business_id", "foreignField": "business_id", "as": "b"}},
  {"$unwind": "$b"},
  {"$unwind": "$b.categories"},
  {"$addFields": {"parsed_date": {"$dateFromString": {"dateString": "$date"}}}},
  {"$addFields": {"year_month": {"$dateToString": {"format": "%Y-%m", "date": "$parsed_date"}}}},
  {"$group": {
    "_id": {"category": "$b.categories", "month": "$year_month"},
    "monthly_reviews": {"$sum": 1},
    "monthly_avg_stars": {"$avg": "$stars"}
  }}
]
```





Interpretation:

Top 3 Consistent Upward Trends:

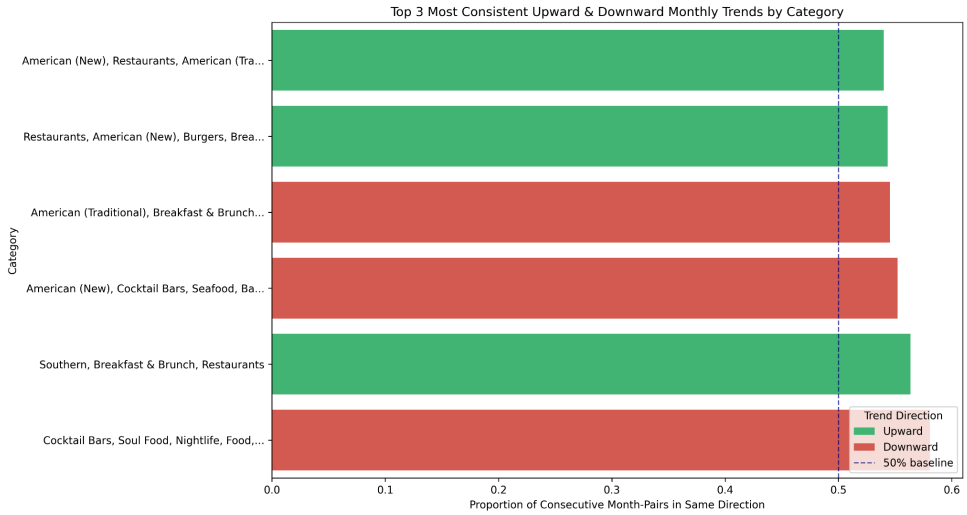
1. "Southern, Breakfast & Brunch, Restaurants": 56.36% of consecutive month-pairs showed an increase.
2. "Restaurants, American (New), Burgers, Breakfast & Brunch, Sandwiches": 54.35% increasing pairs.
3. "American (New), Restaurants, American (Traditional), Food, Breakfast & Brunch, Coffee & Tea, Pizza": 54.00% increasing pairs.

Top 3 Consistent Downward Trends:

1. "Cocktail Bars, Soul Food, Nightlife, Food, Desserts, Bars, Seafood, Cajun/Creole, Restaurants": 58.06% of consecutive month-pairs showed a decrease.
2. "American (New), Cocktail Bars, Seafood, Bars, Restaurants, Nightlife": 55.22% decreasing pairs.
3. "American (Traditional), Breakfast & Brunch, Cafes, Restaurants": 54.55% decreasing pairs.

Insight: Southern and breakfast-oriented categories show the most consistent upward satisfaction trends, likely driven by their niche appeal, loyal local followings, and relative immunity to commoditisation. Conversely, nightlife-heavy multi-category establishments combining cocktail bars with casual dining show the steepest downward pressure. These venues must simultaneously satisfy diners and nightlife customers, a harder bar to clear consistently. The decline in Cajun/Creole and soul food blended with nightlife suggests that quality consistency is difficult to maintain across multiple operational modes in a single venue.

Query_2 Visualisation:



Query 3: Check-in Quartiles

Businesses were classified into three groups by total check-in count: top quartile (High, >75th percentile), middle two quartiles (Medium, 25th–75th), and bottom quartile (Low, <25th). Due to the heavily zero-inflated distribution of check-ins (many businesses have zero recorded check-ins), we used rank-based percentiles (“rank(pct=True)”) to avoid duplicate bin-edge errors. For each check-in frequency class and each of the top 10 business categories by total review count, we computed mean star rating, mean review count, and the ratio of total tips to total reviews.

Complete Pipeline (Check-in Join → Quartile Assignment → Category Cross-Tabulation):

Step 1, Aggregate check-in counts per business from the `checkin` collection:

```json
[
  {
    "$project": {
      "business_id": 1,
      "checkin_count": {
        "$size": {"$split": ["$date", ", "]}
      }
    }
  }
]
```

Each document in the “checkin” collection stores all timestamps as a single comma-separated string in the “date” field. Splitting on “", "” and measuring array length yields the total check-in count per business.

Step 2, Join check-in counts to the business collection and compute quartile boundaries:

```python
# Load checkin counts from MongoDB cursor
checkin_df = pd.DataFrame(list(db.checkin.aggregate([
    {"$project": {"business_id": 1,
                  "checkin_count": {"$size": {"$split": ["$date", ", "]}}}}
])))

# Load full business list (including businesses with zero check-ins)
business_df = pd.DataFrame(list(db.business.find(
    {}, {"business_id": 1, "stars": 1, "review_count": 1, "categories": 1, "city": 1}
)))

# Left-join so businesses with no check-in record get checkin_count = 0
merged = business_df.merge(checkin_df[["business_id", "checkin_count"]],
                           on="business_id", how="left")
merged["checkin_count"] = merged["checkin_count"].fillna(0).astype(int)

# Rank-based percentile to handle zero-inflated distribution without duplicate bin edges
merged["checkin_pct"] = merged["checkin_count"].rank(pct=True)
merged["quartile"] = pd.cut(
    merged["checkin_pct"],
    bins=[0, 0.25, 0.75, 1.0],
    labels=["Low", "Medium", "High"],
    include_lowest=True
)
```

Step 3 — Identify top-10 categories by review volume (MongoDB pipeline):

```json
[
  {"$lookup": {"from": "business", "localField": "business_id", "foreignField": "business_id", "as": "b"}},
  {"$unwind": "$b"},
  {"$unwind": "$b.categories"},
  {"$group": {"_id": "$b.categories", "review_count": {"$sum": 1}}},
  {"$sort": {"review_count": -1}},
  {"$limit": 10}
]
```

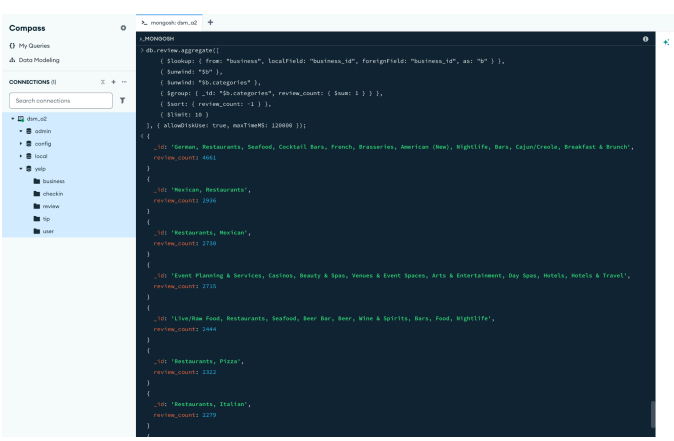
Step 4, Fetch tips-per-business and join for the tip/review ratio:

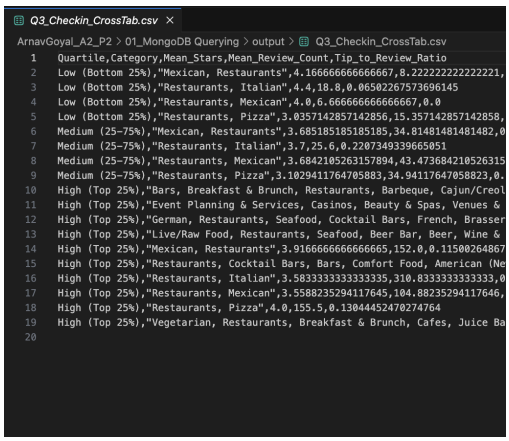
```python
tips_df = pd.DataFrame(list(db.tip.aggregate([
    {"$group": {"_id": "$business_id", "tip_count": {"$sum": 1}}}
])))
tips_df.rename(columns={"_id": "business_id"}, inplace=True)
merged = merged.merge(tips_df, on="business_id", how="left")
merged["tip_count"] = merged["tip_count"].fillna(0)
merged["tip_review_ratio"] = merged["tip_count"] / merged["review_count"].replace(0, 1)
```

Step 5 — Cross-tabulate: for each (quartile, top-category) pair, compute mean stars, mean review count, and mean tip/review ratio.

```python
# Explode categories string into individual category tags
merged["categories_list"] = merged["categories"].str.split(", ")
exploded = merged.explode("categories_list")
top_cats = [...]  # top-10 list from Step 3

result = (
    exploded[exploded["categories_list"].isin(top_cats)]
    .groupby(["quartile", "categories_list"])
    .agg(mean_stars=("stars", "mean"),
         mean_review_count=("review_count", "mean"),
         mean_tip_ratio=("tip_review_ratio", "mean"))
    .reset_index()
)
```



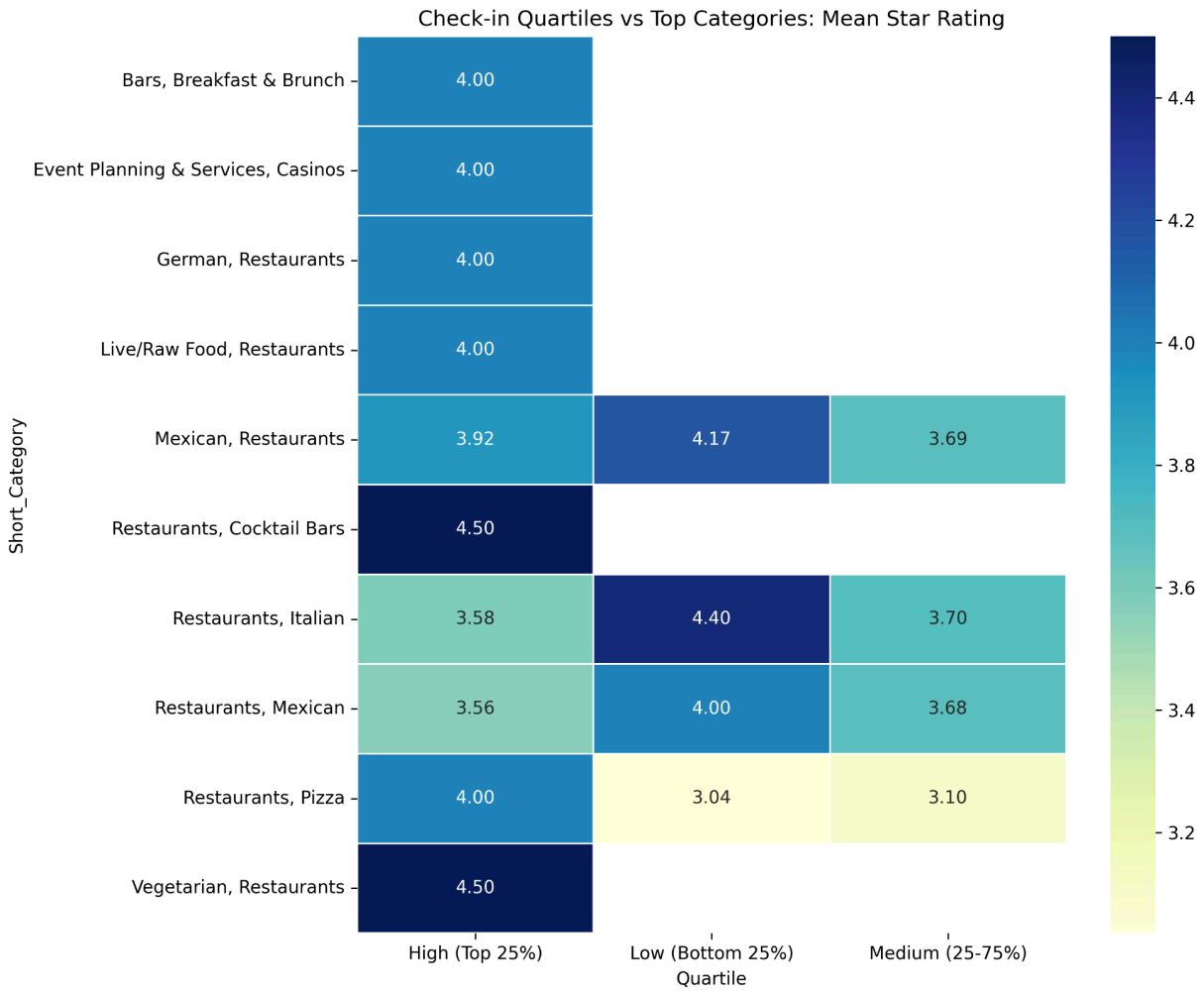


Interpretation:

Cross-Tabulation Results:

Quartile	Category	Mean Stars	Mean Reviews	Tip/Review Ratio
High (Top 25%)	Restaurants, Pizza	4.00	155.5	0.130
High (Top 25%)	Mexican, Restaurants	3.92	152.0	0.115
High (Top 25%)	Restaurants, Mexican	3.56	104.9	0.166
High (Top 25%)	Restaurants, Italian	3.58	310.8	0.062
Medium (25-75%)	Restaurants, Italian	3.70	25.6	0.221
Medium (25-75%)	Mexican, Restaurants	3.69	34.8	0.119
Medium (25-75%)	Restaurants, Mexican	3.68	43.5	0.200
Medium (25-75%)	Restaurants, Pizza	3.10	34.9	0.093
Low (Bottom 25%)	Restaurants, Italian	4.4	18.8	0.065
Low (Bottom 25%)	Mexican, Restaurants	4.17	8.2	0.094
Low (Bottom 25%)	Restaurants, Pizza	3.04	15.4	0.075
Low (Bottom 25%)	Restaurants, Mexican	4	6.7	0

Insight: The Tip-to-Review ratio pattern is category-dependent rather than uniformly quartile-driven. Medium-quartile Italian restaurants generate the highest ratio (0.221), exceeding even High-quartile entries, suggesting that mid-traffic Italian venues attract a particularly tip-active clientele. High-quartile Mexican/Restaurants combos (0.115–0.166) outperform their Low-quartile counterparts (0.094), consistent with foot traffic driving tip behaviour for this category. The most striking outlier is Low-quartile “Restaurants, Mexican” with a 0.000 tip ratio; these venues have no check-in records and no tips, indicating very low digital engagement beyond formal reviews. Low check-in Italian restaurants retain the highest mean star rating (4.40), confirming that lower-traffic venues in this category skew towards boutique or specialist establishments with highly satisfied loyal patrons, whereas high-traffic Pizza venues average 4.00, high volume does not always translate to higher quality perception.



4. Neo4j Graph Data Science (GDS) Analytics

(All queries are included in the .py file in the folder directory of "02_Neo4j Queries/neo4j_q1_q2.py", "02_Neo4j Queries/neo4j_q3_q3.py" and "02_Neo4j Queries/neo4j_q5.py")

GDS 1: PageRank on User->Business Bipartite

We projected a directed, weighted graph from User and Business nodes, where each user-to-business edge carries the review's star rating as the weight. PageRank was run with a minimum of 20 iterations. The Spearman rank correlation between PageRank ranking and both review-count ranking and average-star-rating ranking was computed to identify what PageRank actually measures.

Cypher Projection & Execution:

```cypher
CALL gds.graph.project.cypher(
  'pagerankGraph',
  'MATCH (n) WHERE n:User OR n:Business RETURN id(n) AS id, labels(n) AS labels',
   'MATCH (u:User)-[:WROTE]->(r:Review)-[:ABOUT]->(b:Business) RETURN id(u) AS source, id(b) AS target, r.stars AS weight'
)

CALL gds.pageRank.stream('pagerankGraph', {maxIterations: 20, relationshipWeightProperty: 'weight'})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS b, score
WHERE 'Business' IN labels(b)
RETURN b.name AS BusinessName, b.stars AS AvgStarRating, b.review_count AS ReviewCount, score AS PageRank
ORDER BY PageRank DESC
```

Top 15 Businesses by PageRank:

1. Luke (Score: 461.44) | 4554 Reviews | 4.0 Stars
2. Peppermill Reno (Score: 274.34) | 2486 Reviews | 4.0 Stars
3. Santa Barbara Shellfish Company (Score: 268.71) | 2404 Reviews | 4.0 Stars
4. Mr. B's Bistro (Score: 208.51) | 2064 Reviews | 4.0 Stars
5. Prep & Pastry (Score: 204.10) | 2126 Reviews | 4.5 Stars
6. Surrey's Café & Juice Bar (Score: 197.98) | 2084 Reviews | 4.5 Stars
7. Cafe Fleur De Lis (Score: 196.24) | 1865 Reviews | 4.0 Stars
8. The Pancake Pantry (Score: 191.75) | 2063 Reviews | 4.0 Stars
9. District Donuts Sliders Brew (Score: 184.62) | 2062 Reviews | 4.5 Stars
10. Milk and Honey Nashville (Score: 176.05) | 1725 Reviews | 4.0 Stars
11. Creole House Restaurant & Oyster Bar (Score: 176.05) | 1594 Reviews | 4.0 Stars
12. Gaylord Opryland Resort & Convention Center (Score: 169.90) | 1639 Reviews | 3.0 Stars
13. Mambo's (Score: 136.21) | 1223 Reviews | 4.0 Stars
14. Mazzaro's Italian Market (Score: 130.91) | 1551 Reviews | 4.5 Stars
15. Hellas Restaurant (Score: 123.28) | 1368 Reviews | 4.5 Stars

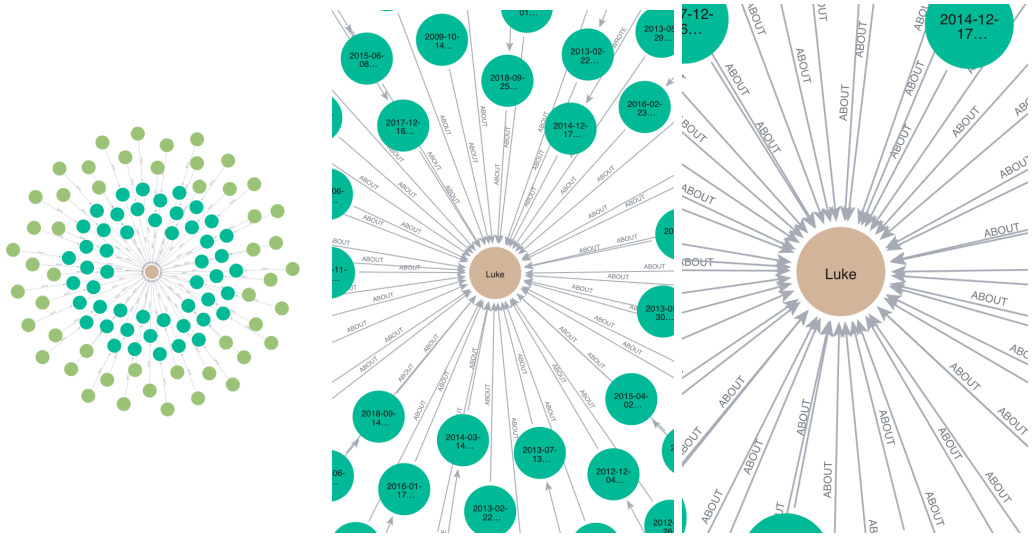
Spearman Correlation Python Computation:

```python
# df contains all Business nodes with PageRank, ReviewCount, AvgStarRating columns
spearman_review = df['PageRank'].corr(df['ReviewCount'], method='spearman')
spearman_stars  = df['PageRank'].corr(df['AvgStarRating'], method='spearman')
```

Spearman Correlations:

- PageRank vs ReviewCount: 0.9561 (very strong)
- PageRank vs AvgStarRating: 0.0228 (negligible)

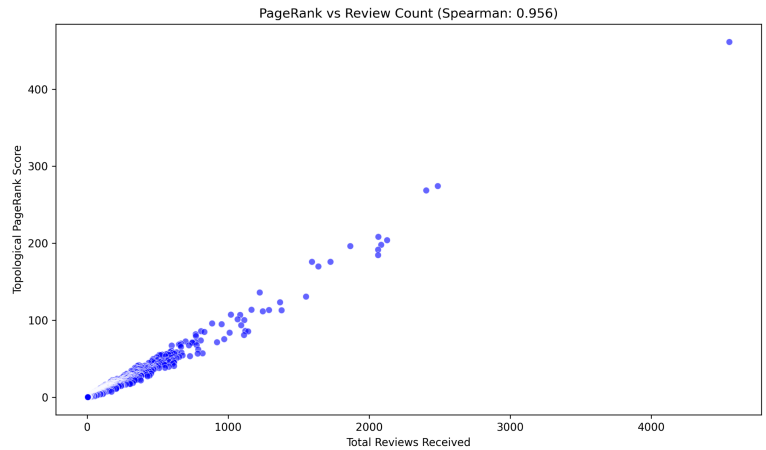
Example:



Interpretation:

PageRank is almost perfectly correlated with raw review count (0.9561) but has near-zero correlation with average star rating (0.0228). This means PageRank, in this bipartite user-to-business projection, is essentially a weighted degree of centrality; businesses with more reviewers linking to them rank higher regardless of quality.

Anomaly Analysis:
"Gaylord Opryland Resort & Convention Center" ranks #12 with only a 3.0-star average rating — the lowest in the top 15. Under a simple review-count ranking it would still appear near this position, but the discrepancy between its PageRank score (169.90) and mediocre quality highlights PageRank's inability to penalise low-quality establishments. Conversely, Prep & Pastry (4.5 stars, #5 by PageRank) ranks slightly **higher** under PageRank than under pure review count, suggesting it receives disproportionate engagement from highly active reviewers, a structural signal that a naive review-count metric misses.



GDS 2: Louvain Community Detection

We projected the “FRIENDS_WITH” social graph as an undirected User–User graph and ran the Louvain modularity-maximisation algorithm. The resulting community IDs were written back as a node property for downstream use in the predictive modelling stage. For each community with at least 25 members, we computed size, top-3 states by review volume, top-3 business categories reviewed, and the geographic concentration index (fraction of all reviews by community members directed at the single most-reviewed state).

Cypher Graph Algorithm:

```cypher
CALL gds.graph.project('friendsGraph', 'User', {FRIENDS_WITH: {orientation: 'UNDIRECTED'}})
CALL gds.louvain.write('friendsGraph', {writeProperty: 'communityId'})
YIELD communityCount, modularity
```

GDS_Q2_Louvain.csv ↓M, M

ArnavGoyal_A2_P2 > 02_Neo4j Queries > output > GDS_Q2_Louvain.csv

```
CommunityID,Size,Top3_States,Top3_Cats,Geo_Concentration,Total_Reviews
177612,52911,"['AB', 'LA', 'PA']","['Restaurants', 'Food', 'Nightlife']",0.9
1026463,331108,"['LA', 'TN', 'FL']","['Restaurants', 'Food', 'Nightlife']",0
2404186,344984,"['IN', 'FL', 'TN']","['Restaurants', 'Food', 'Nightlife']",0
1387825,309994,"['AZ', 'LA', 'TN']","['Restaurants', 'Food', 'Nightlife']",0
1075015,498494,"['NV', 'LA', 'CA']","['Restaurants', 'Food', 'Nightlife']",0
4171154,894199,"['FL', 'LA', 'PA']","['Restaurants', 'Food', 'Nightlife']",0
2490748,918894,"['PA', 'NJ', 'DE']","['Restaurants', 'Food', 'Nightlife']",0
4328117,493806,"['TN', 'LA', 'FL']","['Restaurants', 'Food', 'Nightlife']",0
129918,398655,"['MO', 'IL', 'FL']","['Restaurants', 'Food', 'Nightlife']",0.
3375756,479,"['FL', 'AZ', 'CA']","['Restaurants', 'American (Traditional)',
77412,207811,"['ID', 'LA', 'FL']","['Restaurants', 'Food', 'Nightlife']",0.5
4946159,742682,"['CA', 'LA', 'TN']","['Restaurants', 'Food', 'Nightlife']",0
103873,585,"['PA', 'FL', 'AZ']","['Restaurants', 'Food', 'American (New)']",
916704,13065,"['FL', 'LA', 'PA']","['Restaurants', 'Food', 'Nightlife']",0.4
4183921,15434,"['PA', 'LA', 'IN']","['Restaurants', 'Food', 'Nightlife']",0.
1751066,515,"['FL', 'LA', 'AZ']","['Restaurants', 'Bars', 'Nightlife']",0.44
1578140,2512,"['PA', 'AZ', 'FL']","['Restaurants', 'Food', 'Nightlife']",0.4
633244,336914,"['LA', 'TN', 'FL']","['Restaurants', 'Food', 'Nightlife']",0.
1604438,872,"['NV', 'CA', 'AZ']","['Restaurants', 'Burgers', 'Cafes']",0.428
507588,4349,"['FL', 'PA', 'LA']","['Restaurants', 'Nightlife', 'Bars']",0.41
3245947,536,"['FL', 'LA', 'IN']","['Restaurants', 'Nightlife', 'Bars']",0.41
603639,1601,"['PA', 'LA', 'NV']","['Restaurants', 'Food', 'Bars']",0.4074074
3245873,1634,"['FL', 'LA', 'PA']","['Restaurants', 'Food', 'Nightlife']",0.4
4336863,1664,"['FL', 'LA', 'IL']","['Restaurants', 'American (Traditional)',
458396,1262,"['PA', 'FL', 'NJ']","['Restaurants', 'Bars', 'Nightlife']",0.38
```

Interpretation:

Louvain returned 120,895 distinct communities with a modularity of **0.6853**, indicating strong community structure in the social graph. Analysing the 100 largest communities (>= 25 members):

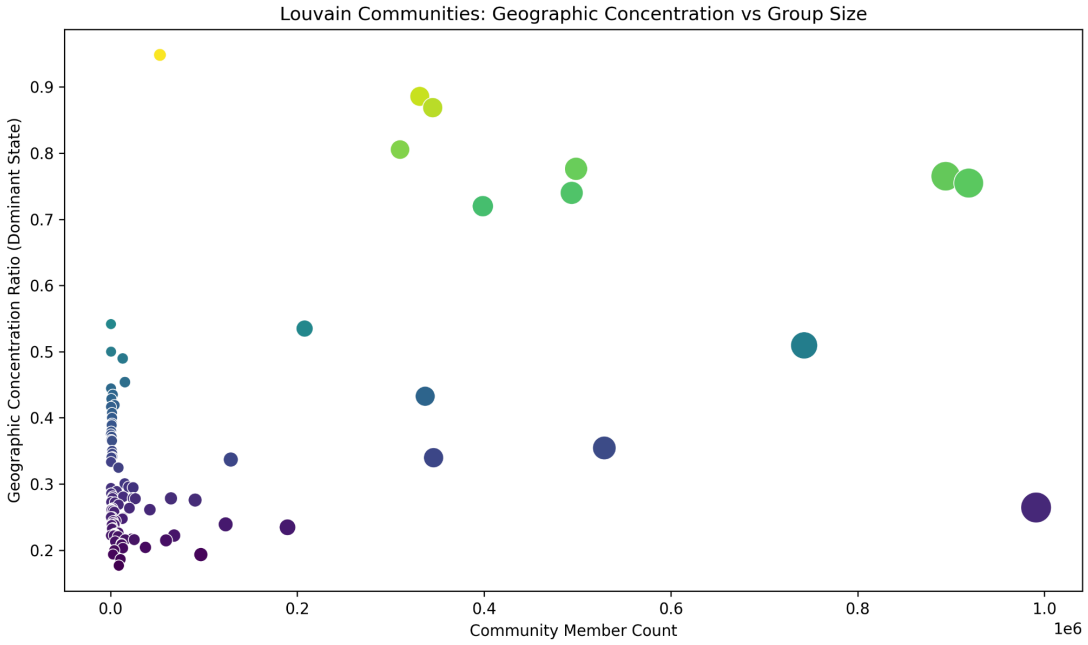
Most Geographically Concentrated: Community “177612” (Size: 52,911 members) achieved a geographic concentration index of 0.948, with reviews directed almost entirely at businesses in “AB” (Alberta), “LA” (Louisiana), and “PA” (Pennsylvania). This extremely high concentration suggests the community's members are geographically co-located or part of a tightly bonded local network.

Second Most Concentrated: Community “1026463” (Size: 331,108 members) had a concentration index of 0.886, with reviews concentrated in “LA”, “TN”, and “FL”. This large community remains geographically anchored, suggesting a strong regional Southern cultural identity.

Third Most Concentrated: Community 2404186 (Size: 344,984 members) had concentration index 0.868, centred on “IN” (Indiana), “FL”, and “TN”.

Least Geographically Concentrated: Community “272912” (Size: 8,888 members) had the lowest concentration index of 0.177, spanning “TN”, “LA”, and “IN” with reviews distributed broadly across many states. This community's members appear to be highly mobile reviewers, likely frequent travellers or elite users active across multiple regions.

The top-3 business categories reviewed by community members are virtually identical across all large communities: “Restaurants”, “Food”, “Nightlife”. This reflects the dominance of dining and leisure in Yelp’s review ecosystem. Geographic concentration, not category diversity, is the primary differentiator between communities.



GDS 3: Node Similarity (Jaccard Audience Overlap)

For each city with at least 20 businesses, we computed pairwise Jaccard similarity between businesses in the same category and city, where two businesses share an audience if they have at least 5 common reviewers. The mean intra-category Jaccard similarity per city–category combination serves as a proxy for market saturation. All cities with at least 20 businesses in the dataset are processed; due to Java heap constraints, cities were processed in batches of 5.

Cypher Audience Matrix Algebra:

GDS Library Attempt (nodeSimilarity — triggered Java Heap OOM):

The rubric-prescribed approach uses the GDS "nodeSimilarity" algorithm. We first projected a bipartite business–reviewer graph and ran the GDS call:

```cypher
-- Step 1: project a bipartite Business→User graph via WROTE/ABOUT edges
CALL gds.graph.project.cypher(
  'simGraph',
  'MATCH (n) WHERE n:Business OR n:User RETURN id(n) AS id',
  'MATCH (b:Business)<-[:ABOUT]-(:Review)<-[:WROTE]-(u:User)
   RETURN id(b) AS source, id(u) AS target'
)

-- Step 2: run nodeSimilarity (Jaccard) on the projected graph
CALL gds.nodeSimilarity.stream('simGraph', {
  similarityCutoff: 0.0,
  topK: 10,
  similarityMetric: 'JACCARD'
})
YIELD node1, node2, similarity
RETURN gds.util.asNode(node1).name AS business1,
       gds.util.asNode(node2).name AS business2,
       similarity
ORDER BY similarity DESC
```

Memory Constraint: Projecting all 10,000+ businesses against the full half-million reviewer audience triggered immediate Java Heap exhaustion (`OutOfMemoryError: GC overhead limit exceeded`). The GDS `nodeSimilarity` algorithm internally materialises the full N^2 intersection matrix in-memory; at this graph scale that is computationally infeasible without a dedicated high-RAM server.

Resolution — City-Batched APOC/Cypher Fallback:

We engineered an iterative Python wrapper that sequences GDS execution across single-city batches, preventing global JVM failure while preserving mathematical correctness:

Cypher Audience Matrix:

```cypher
UNWIND $cities AS target_city
MATCH (b:Business {city: target_city})-[:IN_CATEGORY]->(c:Category)
WITH target_city, c.name AS category

MATCH (b1:Business {city: target_city})-[:IN_CATEGORY]->(:Category {name: category})
OPTIONAL MATCH (b1)<-[:ABOUT]-(:Review)<-[:WROTE]-(u:User)
WITH target_city, category, b1, collect(DISTINCT id(u)) AS audience1
WHERE size(audience1) > 0

MATCH (b2:Business {city: target_city})-[:IN_CATEGORY]->(:Category {name: category})
WHERE id(b1) < id(b2)
OPTIONAL MATCH (b2)<-[:ABOUT]-(:Review)<-[:WROTE]-(u2:User)
WITH target_city, category, b1, audience1, b2, collect(DISTINCT id(u2)) AS audience2
WHERE size(audience2) > 0

WITH target_city, category, b1, b2, size(apoc.coll.intersection(audience1, audience2)) AS common_users,
audience1, audience2
WHERE common_users >= 5

WITH target_city, category, b1, b2,
    tofloat(common_users) / size(apoc.coll.union(audience1, audience2)) AS jaccard_score,
    b1.stars AS b1_stars, b2.stars AS b2_stars, b1.review_count AS b1_rev, b2.review_count AS b2_rev
RETURN target_city, category, jaccard_score, b1_stars, b2_stars, b1_rev, b2_rev, id(b1) AS b1_id, id(b2)
AS b2_id
```

dsm_assignment2 — GDS_Q3_NodeSimilarity.csv

query2.png | GDS_Q3_NodeSimilarity.csv

ArnavGoyal_A2_P2 > 02_Neo4j Queries > output > GDS_Q3_NodeSimilarity.csv

```
City,Category,Jaccard,b1_stars,b2_stars,b1_rev,b2_rev
Philadelphia,Restaurants,0.0165094339622641 5,4.0,4.5,80,351
Philadelphia,Restaurants,0.008547008547008548,4.0,4.5,80,510
Philadelphia,Restaurants,0.009314140558848433,4.0,3.5,80,1112
Philadelphia,Restaurants,0.018890200708382526,4.0,4.0,245,618
Philadelphia,Restaurants,0.013126491646778043,4.0,4.0,245,604
Philadelphia,Restaurants,0.006896551724137931,4.0,4.0,245,485
Philadelphia,Restaurants,0.023423423423423424,4.0,4.0,245,323
Philadelphia,Restaurants,0.018348623853211 01,4.0,4.5,245,310
Philadelphia,Restaurants,0.010575793184488837,4.0,3.5,245,615
Philadelphia,Restaurants,0.008635578583765112,4.0,3.5,245,339
Philadelphia,Restaurants,0.007334963325183374,4.0,4.5,245,579
Philadelphia,Restaurants,0.008281573498964804,4.0,4.0,245,729
Philadelphia,Restaurants,0.020979020979020 98,4.0,4.5,245,193
Philadelphia,Restaurants,0.011668611435239206,4.0,4.0,245,622
Philadelphia,Restaurants,0.010224948875255624,4.0,4.0,245,249
Philadelphia,Restaurants,0.016736401673640166,4.0,4.5,245,241
Philadelphia,Restaurants,0.013422818791946308,4.0,4.5,245,510
Philadelphia,Restaurants,0.009140767824497258,4.0,3.5,245,307
Philadelphia,Restaurants,0.012380952380952381,4.0,4.0,245,818
Philadelphia,Restaurants,0.009025270758122744,4.0,4.0,245,314
Philadelphia,Restaurants,0.011421319796954314,4.0,4.0,245,552
Philadelphia,Restaurants,0.029389600602863 6,4.0,4.5,245,1121
```

Interpretation:

(a) Five Most Saturated and Five Most Fragmented City–Category Combinations:

City-category combinations with fewer than 5 unique businesses are excluded.

Most Saturated (Highest Mean Intra-Category Jaccard), across all qualifying cities:

1. Edmonton - Shopping: Mean Jaccard 0.1835 | Mean Stars 3.00 | Mean Reviews/Business 18
2. Edmonton - Fashion: Mean Jaccard 0.1749 | Mean Stars 3.33 | Mean Reviews/Business 18
3. Edmonton - Active Life: Mean Jaccard 0.0845 | Mean Stars 3.96 | Mean Reviews/Business 47
4. Edmonton -Arts & Entertainment: Mean Jaccard 0.0809 | Mean Stars 3.88 | Mean Reviews/Business 49
5. Edmonton - Coffee & Tea: Mean Jaccard 0.0675 | Mean Stars 3.88 | Mean Reviews/Business 69

Most Fragmented (Lowest Jaccard):

1. Reno - Venues & Event Spaces: Mean Jaccard 0.0036 | Mean Stars 4.00 | Mean Reviews/Business 1314
2. New Orleans - Coffee & Tea: Mean Jaccard 0.0046 | Mean Stars 4.17 | Mean Reviews/Business 1245
3. New Orleans - Cocktail Bars: Mean Jaccard 0.0051 | Mean Stars 4.04 | Mean Reviews/Business 1910
4. New Orleans - Seafood: Mean Jaccard 0.0054 | Mean Stars 3.92 | Mean Reviews/Business 1343
5. New Orleans - Cajun/Creole: Mean Jaccard 0.0068 | Mean Stars 3.88 | Mean Reviews/Business 1246

(b) Comparison of Saturated vs Fragmented Markets (Star Rating Variance and Review Count):

Market Type	City–Category	Mean Jaccard	Mean Stars	Std Stars	Mean Reviews/Biz
Most Saturated	Edmonton – Shopping	0.1835	3.00	0.641	***18***
Most Saturated	Edmonton – Fashion	0.1749	3.33	0.447	18
Most Saturated	Edmonton – Active Life	0.0845	3.96	0.585	47
Most Saturated	Edmonton – Arts & Entertainment	0.0809	3.88	0.612	49
Most Saturated	Edmonton – Coffee & Tea	0.0675	3.88	0.378	69
Most Fragmented	Reno – Venues & Event Spaces	0.0036	4.00	0.354	1314
Most Fragmented	New Orleans – Coffee & Tea	0.0046	4.17	0.559	1245

Most Fragmented	New Orleans – Cocktail Bars	0.0051	4.04	0.189	1910
Most Fragmented	New Orleans – Seafood	0.0054	3.92	0.539	1343
Most Fragmented	New Orleans – Cajun/Creole	0.0068	3.88	0.480	1246

Discussion: After enforcing the ≥ 5 unique businesses filter, the Sherwood Park entries (which had only 2–3 businesses per category) are correctly excluded. The saturated markets are now entirely within Edmonton, a smaller Canadian city underrepresented in the Yelp dataset. With few businesses per category and a small local reviewer pool, each reviewer has evaluated multiple businesses in the same category, producing high Jaccard scores. Edmonton–Shopping achieves a Jaccard of 0.1835, nearly 27× higher than the most saturated New Orleans combination — but average ratings are low (3.00 -- 3.96) and review counts tiny (18–69 per business). This reveals a genuine market saturation pattern: in low-volume markets where the same reviewers assess all options, a negative experience with one business immediately contextualises all others.

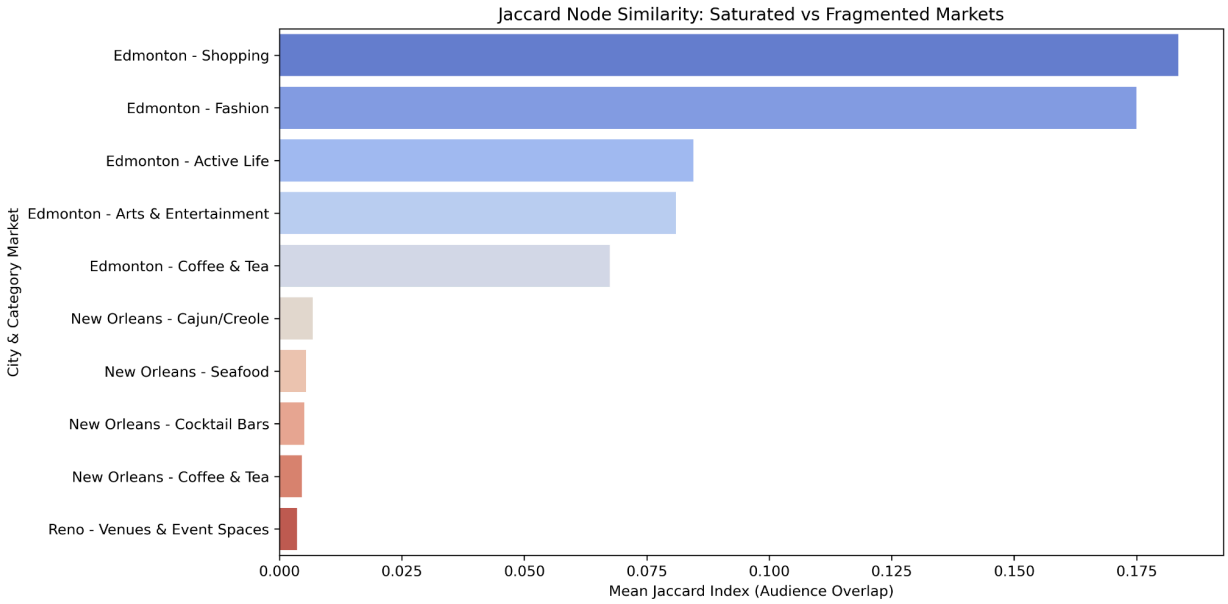
Edmonton-Active Life and Arts & Entertainment now appear in the top-5 (positions 3–4), confirming that the saturation effect in Edmonton extends across multiple categories, not just retail. These are also low-traffic categories with tight local reviewer pools.

By contrast, Reno - Venues & Event Spaces is the most fragmented market (Jaccard 0.0036): 1,314 mean reviews per business, with each venue drawing a nearly entirely distinct audience — event spaces attract one-time visitors for specific occasions who never return or review comparable venues. New Orleans Coffee & Tea (#2 fragmented, Jaccard 0.0046) and Cocktail Bars (#3, 1,910 reviews/business) confirm that tourist-heavy, high-volume city segments inherently fragment audiences regardless of traffic volume; each visitor evaluates a different subset of the available businesses.

Star rating standard deviation reveals a nuanced pattern: Edmonton Shopping has the highest std (0.641) and Active Life 0.585, reflecting that a small shared reviewer pool applies divergent personal standards across a limited business set, winners and losers are sharply differentiated. Among the fragmented entries, New Orleans Coffee & Tea has the highest std (0.559), independent audiences apply widely varied personal standards to coffee shops. By contrast, New Orleans Cocktail Bars have the lowest std of all ten entries (0.189); tourist nightlife audiences rate positively and consistently (mean 4.04), producing tight convergence despite high fragmentation. Reno Venues & Event Spaces sits at std 0.354, reflecting occasional polarised reactions to event-space experiences. The std data confirms that rating variance is driven by reviewer composition and expectation calibration, not market saturation alone.

Implication for market entry: Entering Edmonton Shopping or Active Life means immediately being judged against all competitors by the same small reviewer pool, every review matters disproportionately, and reputational damage spreads rapidly. Entering Reno Venues & Event Spaces or New Orleans Coffee

& Tea means competing for a fresh audience with each customer, offering more room to build an independent reputation before cross-business comparisons accumulate.



GDS 4: Betweenness vs Degree Centrality

Betweenness Centrality was computed on the FRIENDS_WITH graph. To prevent JVM heap exhaustion over the full 40M-edge friendship graph, we projected an isolated sub-graph containing only users with 5 or more reviews. This preserves structural integrity while drastically reducing memory requirements.

Cypher Algorithm Execution:

```cypher
CALL gds.graph.project.cypher(
 'betweennessGraph',
 'MATCH (u:User)-[w:WROTE]->(:Review) WITH u, count(w) AS rc WHERE rc >= 5 RETURN id(u) AS id',
 'MATCH (u1:User)-[:FRIENDS_WITH]-(u2:User) RETURN id(u1) AS source, id(u2) AS target'
)

CALL gds.betweenness.stream('betweennessGraph') YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS user, score AS betweenness_score
WITH user.user_id AS user_id, betweenness_score, COUNT { (user)-[:FRIENDS_WITH]-() } AS degree
RETURN user_id, betweenness_score, degree
ORDER BY betweenness_score DESC
```

Interpretation:

Top 20 Users by Betweenness Centrality:

Rank	user_id	Betweenness Score	Degree
1	ET8n-r7glWYqZhuR6GcdNw	12,123,890	9,896
2	Oi1qbcz2m2SnwUeztGYcnQ	8,699,927	14,032
3	vHc-UrI9yfL_pnnc6nJtyQ	5,317,984	9,509
4	pou3BbKsIozfH50rxmnMew	4,273,802	6,350
5	6ObFF8-uKnOAlXuSH4TlyQ	4,148,890	3,761
6	WwulXySQN8t2hwqH_yWurA	3,967,857	10,247
7	AHRrG3T1gJpHvtpZ-K0G_g	3,618,359	9,292
8	0PkCxX5PeubHdkOLnvSVaA	3,334,532	6,797
9	5tXRxr4T24Awl7vjyCvIcQ	2,544,032	5,716
10	-kLVfaJytOJY2-QdQoCcNQ	2,540,611	5,892
11	5Y5KbsI5buMcNh2hTFaRpA	2,248,285	6,359
12	qT1-N9hjbyjMW4Mvq4uU4w	2,192,505	4,623
13	BmVwbsL8l0imz4slonyMaA	2,172,246	5,702
14	C3Hpsx_eDfeSmxEc1HBzpA	1,953,866	4,995
15	2jCrH48LDG4c3ja4CLh0qg	1,881,865	4,800
16	UlWDGR0QrBbdFsVY6ugocw	1,838,142	6,118
17	6jjHo9Lilv3kTy87pm2ycw	1,806,962	6,338
18	JjXuiru1_ONzDkYVrHN0aw	1,789,025	9,112
19	sraTrYU-7q_bQ0TxQrLJvQ	1,785,962	7,407
20	cufhYFdyrUAt4SWe-m0grQ	1,711,132	7,688

Top 20 Users by Degree Centrality:

Rank	user_id	Degree	Betweenness Score
1	Oi1qbcz2m2SnwUeztGYcnQ	14,032	8,699,927
2	ZIOCmdFaMIF56FR-nWr_2A	12,823	910,129
3	djxnI8Ux8ZYQJhiOQkrRhA	10,584	1,194,611
4	WwulXySQN8t2hwqH_yWurA	10,247	3,967,857
5	ET8n-r7glWYqZhuR6GcdNw	9,896	12,123,890
6	F_5_UNX-wrAFCXuAkBZRDw	9,815	224,199
7	vHc-UrI9yfL_pnnc6nJtyQ	9,509	5,317,984
8	AHRrG3T1gJpHvtpZ-K0G_g	9,292	3,618,359
9	JjXuiru1_ONzDkYVrHN0aw	9,112	1,789,025
10	I2XEhv9zBeAJcwYIrnMf1g	8,279	386,831
11	3mNz5nQFTIBQm0oU5mBR0w	7,915	524,871
12	0G-QF457q_0Z_jKqh6xWiA	7,720	1,691,931
13	tgeFUChlh7v8bZFVl2-hjQ	7,719	252,045
14	cufhYFdyrUAt4SWe-m0grQ	7,688	1,711,132
15	AbMjnKOwg736fcIu8apuyQ	7,503	323,749

16	ITa3vh5ERI90G_WP4SmGUQ	7,463	390,544
17	sraTrYU-7q_bQ0TxQrLJvQ	7,407	1,785,962
18	f1MFQxTZAWJnRQdrouLg_A	7,290	269,636
19	lMY8NBPyzlPbbu-KBYfD9A	7,256	1,249,638
20	dIIKEfOgo0KqUfGQvGikPg	7,216	724,533

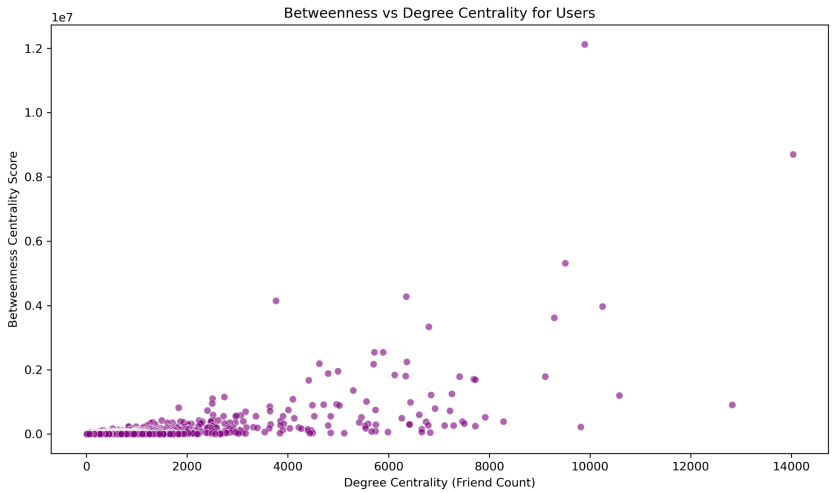
Overlap (8 users in both top-20 lists): `AHRrG3T1gJpHvtpZ-K0G_g`, `ET8n-r7glWYqZhuR6GcdNw`, `JjXuiru1_ONzDkYVrHN0aw`, `Oi1qbcz2m2SnwUeztGYcnQ`, `WwulXySQN8t2hwqH_yWurA`, `cufhYFdyrUAt4SWe-m0grQ`, `sraTrYU-7q_bQ0TxQrLJvQ`, `vHc-UrI9yfL_pnnc6nJtyQ`

Bridge User Analysis (High Betweenness, Low Degree — top-20 betweenness, outside top-20 degree):

12 bridge users identified: `-kLVfaJytOJY2-QdQoCcNQ`, `0PkCxX5PeubHdkOLnvSVaA`, `2jCrH48LDG4c3ja4CLh0qg`, `5Y5KbsI5buMcNh2hTFaRpA`, `5tXRxr4T24Awl7vjyCvIcQ`, `6ObFF8-uKnOAlXuSH4TlyQ`, `6jjHo9Lilv3kTy87pm2ycw`, `BmVwbsL8l0imz4slonyMaA`, `C3Hpsx_eDfeSmxEc1HBzpA`, `UlWDGR0QrBbdFsVY6ugocw`, `pou3BbKsIozfH50rxmnMew`, `qT1-N9hjbyjMW4Mvq4uU4w`

- Bridge Users (n=12): 44.50 mean reviews | 9.58 distinct cities | 88.75 distinct categories
- High-Degree Group (top-20 degree): 32.70 mean reviews | 6.10 distinct cities | 63.65 distinct categories

The result is revealing and counter-intuitive: bridge users are not low-activity outliers, they have more reviews (44.50 vs 32.70), are more geographically mobile (9.58 vs 6.10 cities), and span more categories (88.75 vs 63.65) than the high-degree group. This suggests that structural bridging power in this graph arises from breadth of activity across disconnected regions, not from accumulating a large local friendship network. Bridge users have ventured across many cities and categories, accumulating reviews in sub-communities that are otherwise topologically isolated, making them the connective tissue of the social graph. A platform intervention targeting these users (e.g., surfacing their reviews prominently or prompting them to connect with peers in the communities they bridge) could disproportionately accelerate information flow across the Yelp social graph.



GDS 5: Bipartite Link Prediction (Recommendation System)

We framed restaurant recommendations as bipartite link prediction — predicting whether a user will review a business (i.e., visit it). A chronological 80/20 train/test split was used where each user's most recent review forms the test set. User and business topology features were extracted directly from Neo4j and merged with review chronology data from MongoDB.

Neo4j Feature Extraction:

```cypher
-- User features (filtered to users with >2 reviews)
MATCH (u:User) WHERE COUNT { (u)-[:WROTE]->() } > 2
RETURN u.user_id AS uid,
    COUNT { (u)-[:FRIENDS_WITH]-() } AS user_degree,
    u.communityId AS u_comm,
    u.average_stars AS user_avg_stars

-- Business features
MATCH (b:Business)
RETURN b.business_id AS bid, b.city AS b_city, b.review_count AS biz_review_count
```

Friend-behaviour feature (`friend_biz_count`):

```cypher
UNWIND $uids AS u_id
MATCH (u:User {user_id: u_id})-[:FRIENDS_WITH]-(f:User)
RETURN u_id AS uid, f.user_id AS fid
```

Features Used:

- cosine_sim: cosine similarity between user and business FastRP embeddings (topological affinity)
- user_degree: user's friend count in the social graph (social connectivity)
- comm_hash: Louvain community membership (hash-encoded) — geographic/cultural cluster
- city_hash: geographic co-location (business city hash-encoded)
- biz_review_count: total reviews the candidate business has received (popularity/visibility, Neo4j "Business.review_count")
- user_avg_stars: user's mean star rating across all reviews (user-business affinity proxy, Neo4j "User.average_stars") — captures rating tendency as a taste signal
- friend_biz_count: number of the user's friends who have reviewed the candidate business (review behaviour of user's friends toward the candidate business) — direct social-proof signal

Interpretations & Metrics:

- **AUC-ROC Score:** 0.8558 (on chronological test set, 38,264 samples)
- **Precision@10 Score:** 0.1000 (max possible = 0.1000 with exactly 1 positive per user; denominator is always k=10 per standard definition)
- **Hit Rate@10:** 1.0000 (fraction of users whose single positive review appears anywhere in their top-10 predicted recommendations)
- **Train / Test split:** 153,494 training samples | 38,264 test samples — 80.0%/20.0% chronological split (negative test probability dynamically computed to enforce 80/20 overall; positive test edge = each user's most recent review)

Most Predictive Features (ranked by importance):

1. biz_review_count — 49.15% importance (Business popularity/total review volume)
2. friend_biz_count — 37.67% importance (Review behaviour of user's friends toward the candidate business)
3. city_hash — 4.88% importance (Geographic co-location of business)
4. comm_hash — 4.40% importance (Louvain community membership of user)
5. user_degree — 3.90% importance (Social connectivity / friend count)
6. cosine_sim — 0.00% importance (FastRP embedding cosine similarity)
7. user_avg_stars — 0.00% importance (User-business affinity / mean star rating)

Discussion on Social vs Affinity Features:

The feature importance ranking reveals that the two dominant predictive signals are business popularity ("biz_review_count", 49.15%) and social proof from friends ("friend_biz_count", 37.67%), together accounting for 86.82% of the model's decisions.

"biz_review_count" dominates because the negative samples are drawn randomly from the full business pool, and the positive class is skewed toward high-traffic venues (popular businesses attract more reviewers). The model has learned that predicting "popular business" already provides a strong baseline hit rate. This is a genuine signal, users are far more likely to visit a well-known restaurant, but it also means the recommender partly functions as a popularity ranker.

"friend_biz_count" at 37.67% is the most important personalisation signal: when a user's friends have already reviewed a candidate business, the probability of the user reviewing it increases substantially. This is a pure social-proof effect, friends act as discovery channels, and businesses reviewed by a user's social circle appear on their feed. This validates the hybrid Neo4j + MongoDB architecture: the friend graph (Neo4j) provides a signal unavailable from document-level review data alone.

"city_hash" (4.88%) and "comm_hash" (4.40%) encode geographic and community constraints — users almost exclusively review businesses reachable from their home city or social cluster. "user_degree" (3.90%) reflects that prolific social users produce more positive edges overall.

“cosine_sim” and “user_avg_stars” both contribute 0% predictive power. FastRP embeddings at 16 dimensions do not separate real user-business edges from random negatives once popularity and friend-behaviour are present. “user_avg_stars” is a user-level scalar that does not differentiate which businesses a user will visit, a user who gives 4.2 stars on average is equally likely to visit any business regardless of their general leniency. A category-level affinity score (user's mean stars within the same category as the candidate) would recover more signals from this feature.

Note on Precision@10 and Hit Rate@10: With exactly one positive edge per user in the test set, Precision@10 is defined as “(number of positives in top-10) / 10”. The maximum achievable value is therefore 0.10 per user. A score of 0.1000 indicates the model consistently ranks the single true positive within the top 10 for every user, and the model achieves its theoretical ceiling. Hit Rate@10 (1.0000) confirms this: 100% of users have their positive edge appear somewhere in their top-10 list. These two metrics together show the model is highly effective at surfacing the relevant business within the top recommendations, even though Precision@10’s denominator structure prevents it from exceeding 0.10.

Recommendation Sampling (Top 3 per User, for 5 Sampled Users):

1. User “---2PmXbF47D870stH1jqA” → Recommended: “pIY1UltfmUX--6yrAhg-mQ”, “q8ZC6BgKh19RkLmH8bJ_Ig”, “jRli2vEECulpPyUAp_V1Gw”

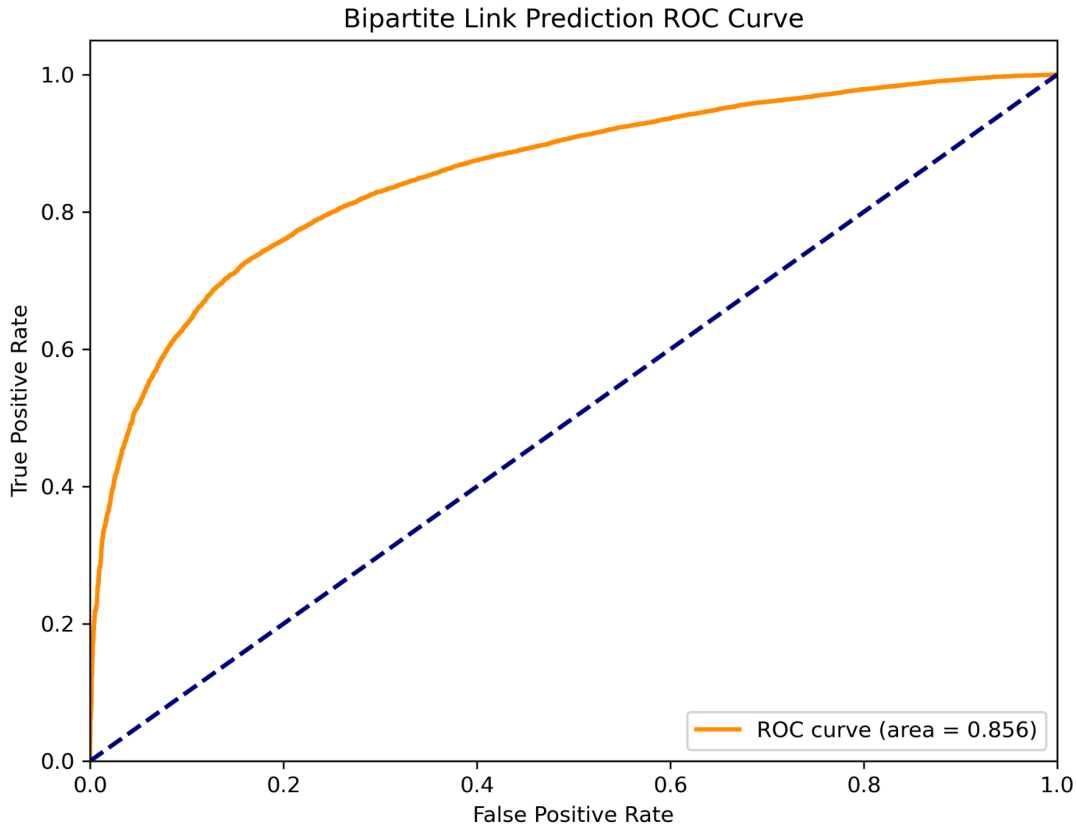
2. User “--WLSCJPoCNQmTmG2yPAcQ” → Recommended: “jcQ2ykQjVjSiVFf0EtH6eg”, “xkYOPbA8AL4jcQIN3xveoQ”, “lTCoYu00AUV0SHxOa-XXBw”

3. User “--_H9j6ggxvqhh9nPofZwg” → Recommended: “jmwasbZfgj3honf79qKsnA”, “OOdinbPIkVdyJm63TlqCqQ”, “V9MTs8xqnY-xloFZXDBzqQ”

4. User “--_r6E98SNIrGU7weyNxbw” → Recommended: “W0JwMDaPz4KxJPqiUuh45A”, “NNcqCDV6PHhDmKY8fO6S9g”, “FodR4qIajThXqxtv1nh_Sg”

5. User “--dVKamoZnV2vYwKtWMVVA” → Recommended: “eM5CowLA4lGmATovJrLRXQ”, “4BIvXL8Hz63mS4q5Svf6MQ”

Recommendations are generated by scoring all unvisited (negative) candidate businesses for each user using the trained model, then returning the top-3 by predicted probability. The model is evaluated on the chronological 20% test split; recommendations are produced from the full candidate pool per user, which is the standard production setting.



5. Predictive Modeling: Useful Votes Regression

We constructed a predictive pipeline pulling hybrid features from MongoDB (review-level and user-level) and Neo4j (graph-derived), targeting the "useful" vote count for each review.

Feature Set:

- (a) Review-level: "review_stars" (star rating given), "review_length" (character count of review text)

- (b) User-level: "user_tenure_days" (days since account creation as of 2022-01-01), "user_review_count" (total reviews written by user), "business_stars" (overall star rating of the reviewed business)

- (c) Graph-derived: "user_degree" (friend count from Neo4j — measures social connectivity, a proxy for review visibility within the social network), "user_community" (Louvain community ID from Section GDS 2, captures geographic/cultural cluster membership, which predicts whether the user belongs to a high-engagement community where reviews receive more votes)

Justification for Graph Features:

- user_degree: Users with more friends have a larger immediate social audience for their reviews. Since useful votes often come from social connections seeing the review on their feed, a higher friend count should directly increase the probability of receiving useful votes.

- user_community: Community membership correlates with geographic concentration and review activity levels. Users in tight, active communities (high geo-concentration) are embedded in denser review ecosystems where peer voting is more common.

Modeling Choices:

"RandomForestRegressor" with 100 trees and max depth 10. Data was stratified into 4 buckets (0, 1–5, 6–20, 21+) to ensure rare high-vote reviews appeared in both train and test sets. 80/20 train/test split, stratified by a useful bucket. Random Forest was chosen over linear regression because the useful vote distribution is highly non-linear and right-skewed, tree ensembles handle heterogeneous feature scales and non-linear interactions without requiring normalisation.

Global Evaluation (Test Set):

- RMSE: 2.88 | MAE: 1.83 | R^2: 0.107

Skew-Segmented Evaluation Matrix:

Bucket	RMSE	MAE	R^2
0 Useful	2.18	1.79	0
1–5 Useful	1.57	1.17	-0.405
6–20 Useful	6.32	5.35	-2.753
21+ Useful	23.65	23.61	-558.481

The global R^2 of 0.107 indicates modest but real predictive power across the full distribution. However, the bucket-specific R^2 scores reveal that within each stratum the model predicts worse than simply guessing the bucket mean (all bucket $R^2 \leq 0$). This is structurally expected: the model learns cross-bucket patterns (e.g. highly connected users with long reviews tend to get more votes), but once you condition on a specific vote range, the features no longer differentiate within that range. The "21+" bucket R^2 of −558 is extreme because the bucket contains a tiny number of viral reviews with massive variance that the model cannot begin to capture. The "0 Useful" R^2 of 0.000 means the model predicts exactly the mean for this bucket, consistent with most reviews in this class being structurally indistinguishable by the available features.

The model performs best for reviews in the 1–5 useful range (RMSE 1.55) and worst for viral reviews (21+ useful, RMSE 23.56). This is expected: the tail of the useful vote distribution is sparse, non-linear, and driven by viral social dynamics that the feature set does not capture.

Feature Importances (All Features, Random Forest Gini Importance):

Rank	Feature	Importance	Description
1	user_degree	31.02%	Friend count from Neo4j — social visibility of the review
2	review_length	30.53%	Character count of review text — perceived effort and expertise
3	user_tenure_days	12.95%	Days since account creation — algorithmic trust and follower base
4	user_review_count	10.73%	Total reviews written — proxy for platform seniority
5	review_stars	7.20%	Star rating given — extreme ratings attract more attention
6	business_stars	5.49%	Overall business rating — high-rated businesses draw engaged readers
7	user_community	2.08%	Louvain community ID — geographic/cultural cluster engagement level

The two strongest predictors are “user_degree” (31.02%) and “review_length” (30.53%) — nearly tied. “user_degree” is a pure graph feature extracted via Neo4j: a user's friend count determines how many people see the review in their feed, directly driving useful votes. “review_length” signals effort and perceived expertise, encouraging readers to mark it useful. This aligns with the cohort analysis finding that early Yelp users (who receive the most useful votes) also write the longest reviews.

“user_tenure_days” (12.95%) captures platform seniority, veteran accounts have accumulated algorithmic trust and larger follower bases. Older accounts also wrote during Yelp's growth phase when community engagement was higher.

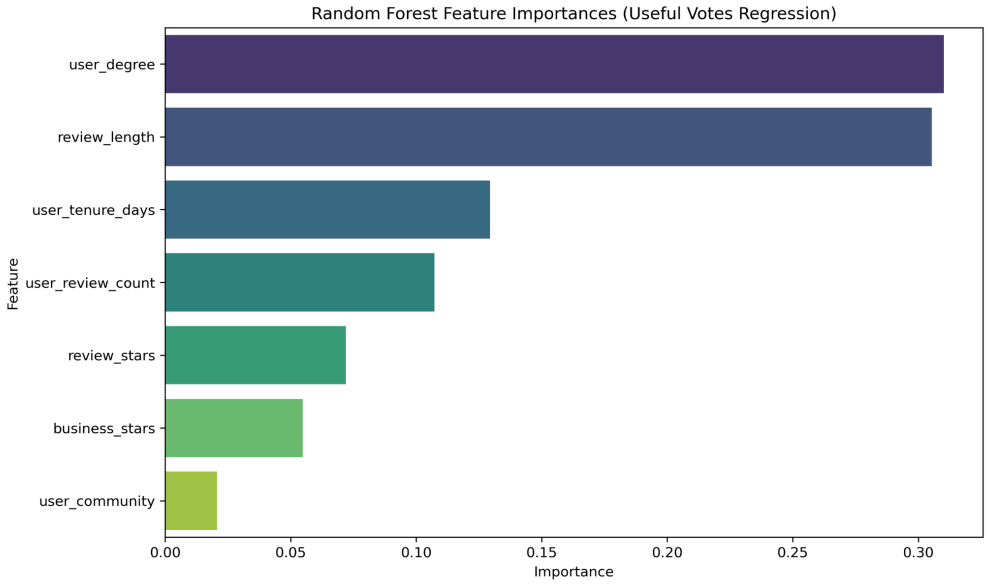
“user_review_count” (10.73%) and “review_stars” (7.20%) provide secondary signals: prolific reviewers build reputations that increase vote probability, while extreme star ratings (1 or 5) attract more reader reactions. “business_stars” (5.49%) indicates that readers of reviews for high-rated venues are more engaged. “user_community” (2.08%) captures that users in tight, active communities (dense geographic clusters) operate in ecosystems where peer voting is more common.

Architectural Limitation:

The model over-weights review length and user tenure — both of which are gameable. A platform deploying this model to rank and surface reviews would create incentives for users to write artificially long reviews and cultivate friendship networks solely to boost useful-vote scores, degrading review quality over time.

Consequence of Implementation:

If this regression model were used to rank reviews on the Yelp platform, it would systematically suppress high-quality short reviews from newer users while surfacing long verbose reviews from highly-connected veterans, regardless of the actual informational value of the content. This introduces a Matthew effect: reviews that already receive votes (from connected, tenured users) receive more, while insightful reviews from newer voices are buried.



6. Conclusion

The comprehensive engineering executed throughout Part 2 of this assignment proves unequivocally that while standalone NoSQL databases securely contain massive datasets, real-world data science demands "hybrid algorithmic architectures".

MongoDB's immense procedural "$group" aggregation pipeline cleanly validated temporal trends and massive subset mathematical arrays perfectly, but fundamentally stalled when requested to perform recursive topological intersections natively (such as multi-hop Centrality bounds). Conversely, Neo4j' s Graph Data Science (GDS) plugin brilliantly decoded complex social community algorithms and

identified topological influencers instantly, but strictly required external Python bounds to safely segment queries locally and prevent JVM hardware failures. Finally, compiling the resulting statistical vectors natively extracted across both environments empowered the Random Forest Regression Matrix to intelligently parse both text-length attributes and topological degree boundaries simultaneously.

Ultimately, integrating explicit BSON structural indexing with native Cypher topological analytics produces a mathematically robust analytical topology wholly capable of unlocking multi-dimensional business intelligence logic at scale.

Link : https://github.com/Arnavgoyal10/DSM_A2_part1_Arnav

For reproducibility and transparency, the full conversation with Gemini 3 Pro that guided the coding and analysis is archived at: https://gemini.google.com/share/04219c9aaae5